



DualCardDLL Specification

DUALi Inc.

Document Version: 1.13

Last Revised Date: 22 JUL 2014

Copyright © 2009 DUALi Inc. All rights reserved. You are strictly prohibited to copy, disclose, distribute, or use this document in part or as a whole for any purposes other than those for which this document is disclosed. This document is copyrighted and contains confidential information and other intellectual property rights of DUALi Inc. Any unauthorized use, copy, disclosure or distribution constitutes infringement of DUALi's intellectual property rights.

DUALi Inc. reserves the right to make changes to its applications or services or to discontinue any application or service at any time without notice. DUALi provides customer assistance in various technical areas, but does not have full access to data concerning the use and applications of customer's products.

Therefore, DUALi assumes no liability and is not responsible for customer applications or software design or performance relating to systems or applications incorporating DUALi products. In addition, DUALi assumes no liability and is not responsible for infringement of patents and/or any other intellectual or industrial property rights of third parties, which may result from assistance provided by DUALi.

Composition of the information in this manual has been done to the best of our knowledge. DUALi does not guarantee the correctness and completeness of the details given in this manual and may not be held liable for damages ensuing from incorrect or incomplete information. Since, despite all our efforts, errors may not be completely avoided, we are always grateful for your useful tips.

We have our development center in South Korea to provide technical support. For any technical assistance can contact our technical support team as below;

Tel: +82 31 213 0074

e-mail : yekwon@duali.com

Revision History

- 2009.02.(Ver. 1.0) : First Release
- 2009.04.(Ver. 1.01): 1.6 DE_ByPass Commnad added
- 2009.04.(Ver. 1.02): 7. ISO 15693
- 2009.05.(Ver. 1.03): 5.4 DEA_ANTICOLL
- 2009.06 (Ver. 1.04): 2.10, 2.11, 5.28, 5.30, 5.31, 6.3, 6.4 Added / Modified
- 2009.07.(Ver. 1.05): 2.11, 2.12, 5.34, 5.35, 5.36 Added
- 2009.12.(Ver. 1.06): Update timeout value
- 2009.12.(Ver. 1.07): Update instruction for WinCE user.
- 2010.08.(Ver. 1.08): Part 9. Fingerprint Command Added
- 2010.12 (Ver. 1.09): Update DE_RWFlash function
- 2011.02.(Ver. 1.10): HTY, DEA_Reset(), LPBYTE Length is 2.
DE-APDU(), add description about third 'LPBYTE data' parameter.
DE-FindCard(), add explanation about option data
- 2012.01.(Ver. 1.11): Part 10. LLC Commands Added,
Part 11. ISO-18092 Commands (optional functions) Added
- 2012.02 (Ver. 1.12): Felica Commands Added
- 2014.07 (Ver. 1.13): Mifare Plus Activation Added

CONTENTS

1	INTRODUCTION	8
2	COMMON API	9
2.1	GETERRMSG	9
2.2	DE_INITPORT	9
2.3	DE_CLOSEPORT	9
2.4	DE_POLLING.....	10
2.5	DE_SERIALPOLLING	10
2.6	DE_ByPASSCOMMAND	10
3	DEVICE CONTROL COMMAND API	12
3.1	DE_RFON.....	12
3.2	DE_RFOFF	12
3.3	DE_RFRESET.....	12
3.4	DE_BUZZEROFF	12
3.5	DE_BUZZERON.....	13
3.6	DE_CHANGEDEVICE.....	13
3.7	DE_GETVERSION.....	13
3.8	DE_CHANGETRXSPEED.....	14
3.9	DE_RWFLASH.....	14
3.10	DE_RfWTX	15
3.11	DE_CONTACTWTX	15
3.12	DE_CONTACTANTI TEARING.....	15
3.13	DE_RFANTI TEARING	16
4	ISO 18092 (FELICA, NFC) API	17
4.1	DEC_TRANSPARENT	17
5	ISO 14443 TYPE B API.....	18
5.1	DEB_TRANSPARENT	18
5.2	DEB_TRANSPARENTCRC.....	18

6	ISO 14443 TYPE A API.....	20
6.1	DEA_RESET.....	20
6.2	DEA_IDLE_REQ	20
6.3	DEA_WAKEUP_REQ.....	20
6.4	DEA_ANTICOLL	21
6.5	DEA_SELECT.....	21
6.6	DEA_AUTH	21
6.7	DEA_HALT.....	22
6.8	DEA_READ.....	22
6.9	INT DEA_WRITE	23
6.10	DEA_INCREMENT.....	23
6.11	DEA_DECREMENT.....	23
6.12	DEA_INC_TRANSFER	24
6.13	DEA_DEC_TRANSFER	24
6.14	DEA_RESTORE	25
6.15	DEA_TRANSFER	25
6.16	DEA_LOADKEY	25
6.17	DEA_AUTHKEY	26
6.18	DEA_REQ_AUTH.....	26
6.19	DEA_REQ_AUTHKEY	27
6.20	DEA_INC_TRANSFER2	27
6.21	DEA_DEC_TRANSFER2	28
6.22	DEA_REQ_AUTHREAD	28
6.23	DEA_REQ_AUTHKEYREAD.....	29
6.24	DEA_REQ_AUTHWRITE.....	29
6.25	DEA_REQ_AUTHKEYWRITE	30
6.26	DEA_REQ_SELECT.....	31
6.27	DEA_ULTRAM_WRITE	31
6.28	DEA_ANTISELLEVEL	32
6.29	DEA_ANTICOLLLEVEL.....	32
6.30	DEA_SELECTLEVEL	33
6.31	DEA_DEVICEINFO	33
6.32	DEA_TRANSPARENT.....	33
6.33	DEA_TRANSPARENTCRC.....	34

6.34	DEA_BITMODE	34
6.35	DEA_BITMODEANTI.....	35
6.36	DEA_BITMODE2	36
6.37	DE_MIFAREPLUS_ACTIVATION.....	36
7	ISO14443 TYPE A/B COMMON API.....	38
7.1	DE_APDU	38
7.2	DE_FINDCARD	38
7.3	DEAB_RwWRITE.....	39
7.4	DEAB_RwREAD	39
8	ISO15693 API	40
8.1	DED_INVENTORY	40
8.2	DED_SELECT.....	40
8.3	DED_READ.....	40
8.4	DED_WRITE	41
8.5	DED_TRANSPARENT.....	41
8.6	DED_EOF	42
9	ISO7816 API	43
9.1	DE_IC_POWERON	43
9.2	DE_IC_PPS	43
9.3	DE_IC_CASE1	43
9.4	DE_IC_CASE2	44
9.5	DE_IC_CASE3	44
9.6	DE_IC_CASE4	45
9.7	DE_IC_POWEROFF.....	45
9.8	DE_T1BYPASS.....	46
9.9	DE_IC_SPEED.....	46
9.10	DE_CARD_APDU	47
10	FELICA COMMANDS	48
10.1	DEC_TRANSPARENT.....	48
10.2	DEC_POLLING_NOENC	48
10.3	DEC_READ_NOENC.....	49

10.4	DEC_WRITE_NoENC	49
10.5	DEC_CONTROLLERNRWAUTH	50
10.6	DEC_FELICAPOLLING	51
10.7	DEC_MUTUALAUTH	51
10.8	DEC_MUTUALAUTH_RWSAM	52
10.9	DEC_READCARDWITHENC	52
10.10	DEC_WRITECARDWITHENC	53
10.11	DEC_READCARDWITHENC_NoMA	54
10.12	DEC_WRITECARDWITHENC_NoMA	55
10.13	DEC_REQUESTSERVICE	55
11	FINGERPRINT COMMAND	57
11.1	DE_FINGERGETIMAGESTART	57
11.2	DE_FINGERGETIMAGEDATA	57
11.3	DE_FINGERENROLL	58
11.4	DE_FINGERVERIFY	58
11.5	DE_FINGERCALIBRATE	59
11.6	DE_FINGERDOWNLOADTEMPLATE	59
12	LLC COMMANDS	60
12.1	LLC_MAC_ACTIVATION	60
12.2	LLC_MAC_DeACTIVATION	60
12.3	LLC_CONNECT_REQUEST	61
12.4	LLC CONNECT RESPONSE	61
12.5	LLC_DISCONNECT_REQUEST	62
12.6	LLC_SEND_RECIEVE_READY	62
12.7	LLC_SEND_SYMMETRY	63
12.8	LLC_SEND_INFORMATION	63
12.9	LLC_SEND_TEXT	63
12.10	LLC_RECEIVE_TEXT	64
13	ISO-18092 COMMANDS (OPTIONAL FUNCTIONS)	66
13.1	DE_NFC_INIT_INITIATOR	66
13.2	DE_NFC_INIT_TARGET	66
13.3	DE_NFC_SENDDATA	67

13.4	DE_NFC_STOP.....	67
13.5	DE_NFC_GETTARGETDATA.....	68
13.6	DE_NFC_SETTARGETDATA	68
13.7	DE_NFC_GETTARGETSTATE.....	69
13.8	DE_NFC_TAG_CMD.....	69
14	RESPONSE CODE DEFINITIONS	76
15	TIME OUT VALUE.....	79

1 Introduction

A. Intended Readership

This manual has been written keeping in mind to the developer who uses the application of DUALi (relating DE-ABx modules /DE-6xx devices)and which mainly acts as a reference manual.

B. Purpose

This document is explained DLL's use method of DUALi Reader and module. We provide 3 relative files, DualCardDll.dll, DualCardDll.lib and DualCardDllheader.h.

In case the develop environment is based on WindowsCE OS, we provide 3files following; DualDlleMbedded.dll, DualDlleMbedded.lib and DualCardDllheader.h

Some protocols which is supportive in Windows OS might not be able to supportive in the WindowsCE OS.

SDK provides Sample program which applied following API, please also refer to Protocol specification for detail protocol information between card and device,

FeliCa™ is registered trademark of SONY corporation
Mifare® and DesFire® are registered trademarks of NXP Semiconductors

2 Common API

2.1 GetErrMsg

int GetErrMsg(int errcode, char* retmsg)

- This function reads error message when it gets response error message using DLL.
- Parameters
 - int errcode[in] : Error code.
 - char* retmsg[out] : Error Message.

2.2 DE_InitPort

int DE_InitPort(int nPort, int nBaud)

- This function carries out Initialization to connect PC(COM / USB) and device.
- Parameters
 - int nPort[in] : Port number.
In case of serial device (RS-232) : COM port number.
Ex) nPort = 1 means COM1.
In case of USB device : from 100~
Ex) nPort = 100 means first connected USB.
nPort = 101 means second connected USB.
 - int nBaud[in] : Baudrate.
In case of serial device (RS-232) : 9600, 19200, 38400, 57600, 115200.
In case of USB device : Doesn't matter
- Return value
Success: nPort
Failure : An error code. For more information, see Response code definition.

2.3 DE_ClosePort

void DE_ClosePort(int nPort)

- This function closes connected device from port.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

2.4 DE_Polling

**int DE_Polling(int nPort, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes,
int timeout = 3000)**

- This function transmits the direct commands to device by USB.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - int timeout = 3000 : default time out setting is 3000. it only used for serial communication.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

2.5 DE_SerialPolling

**int DE_SerialPolling(int nPort, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes,
int timeout = 3000)**

- This function is for transmitting the command to serial device directly.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - int timeout = 3000 : default time out is 3000. Only use for serial communication.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

2.6 DE_ByPassCommand

**int DE_ByPassCommand(int nPort, int datalen, BYTE Cmd, LPBYTE data, LPINT outlen,
LPBYTE lpRes, int timeout = 3000)**

- This function transmits the direct command which is divided Command code/data.

- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - Int datalen[in] : Length of LPBYTE data.
 - BYTE Cmd : Command.
 - LPBYTE data[in] : Command data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - int timeout = 3000 : default time out is 3000. Only use for serial communication.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

3 Device control command API

3.1 DE_RFOn

int DE_RFOn(int nPort)

- This function activates RF field of device of selected port.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

3.2 DE_RFOff

int DE_RFOff(int nPort)

- This function deactivates RF field of device of selected port.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

3.3 DE_RFReset

int DE_RFReset(int nPort)

- This function resets the selected device (Off the RF field and On again).
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
- Return value

Success: DE_OK

Failure : An error code. For more information, see Response code definition.

3.4 DE_BuzzerOff

int DE_BuzzerOff(int nPort)

- This function deactivates device buzzer.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition(page 36..).

3.5 DE_BuzzerOn

int DE_BuzzerOn(int nPort)

- This function activates device buzzer.
- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

3.6 DE_ChangeDevice

int DE_ChangeDevice(int nPort, LPBYTE mode, int Inqflag)

- This function is for change operation mode of connected device.(In case of DE-620, user can easily change the PC/SC & Vendor mode using DualCard software without firmware change)
- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)

- LPBYTE mode[in] : 0x01 = Vendor mode.

- 0x02 = PC/SC mode.

- 0x03 = PCSC+SAM PCSC mode.

- Int Inqflag[in] : 1 = Mode change.

- 0 = Mode inquiry

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition(page 36..).

3.7 DE_GetVersion

int DE_GetVersion(int nPort, LPINT outlen, LPBYTE lpRes)

- This function gets Firmware version of device.
- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)

- LPINT outlen[out] : Length of LPBYTE lpRes.

- LPBYTE lpRes[out] : Response data.

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition(page 36..).

3.8 DE_ChangeTRXSpeed

int DE_ChangeTRXSpeed(int nPort, BYTE Trxspd)

- This function is for change communication speed between device and IC card. It can only change the device communication speed.

- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- BYTE Trxspd[in] : 0xH₁ H₂ (H₁ : TX Speed nibble, H₂ : RX Speed nibble)
H – 0(106K), 1(212K), 2(424K), 3(847K)
Ex) 0x12(TX : 212K, RX : 424K)

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

3.9 DE_RWFlash

Int DE_RWFlash(int nPort, BYTE flag, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes,BYTE offset,BYTE toread)

- This function is for reads or writes user data from flash memory of device.

- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- BYTE flag[in] : 0x00 = Read data, 0xFF = write data.
- int datalen[in] : Length of LPBYTE data.
- LPBYTE data[in]: If BYTE flag is 0x00(Read) : none.
If BYTE flag is 0xFF(Write) : n byte data(Max : 128 byte).
- LPINT outlen[out]: Length of LPBYTE lpRes.
- LPBYTE lpRes[out]: Response data.
- BYTE offset[in] : position to write or read
- BYTE toread[in]: Length to read (if flag is 0xff, set to 0 please)

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition .

3.10 DE_RfWTX

Int DE_RfWTX(int nPort, BYTE Wtx)

- Device regards there're no tag or Error when it sends command to card but not getting any response. Some cases, it needs to extend this waiting time.
- This Function is for extends waiting time of response from Contactless cards. Maximum FWT(9.897s at FWI=15) waiting time can be extended. During card activation sequence, this value will be initialized. This command has to be set up again after initialization.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE Wtx[in] : Extension value.(See. 10. Time Out value).
$$\text{Actual waiting time} = (Wtx + 1) \times \text{extension value}$$
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition .

3.11 DE_ContactWTX

Int DE_ContactWTX(int nPort, BYTE Slot, BYTE Wtx)

- This Function is for extends waiting time of response from Contact cards. Maximum FWT(9.897s at FWI=15)waiting time can be extended. During card activation sequence, this value will be initialized. This command has to be set up again after initialization.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE Slot[in] : Contact slot number.
 - BYTE Wtx[in] : Extension value.(See. 10. Time Out value)..
$$\text{Actual response waiting time} = Wtx \times 2\text{sec.}$$
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition .

3.12 DE_ContactAntiTearing

int WINAPI DE_ContactAntiTearing(int nPort, BYTE AtiH, BYTE AtiM, BYTE AtiL)

- This Function is for deactivate contact card after designated time in command data. The countdown starts after execute this command. Maximum execution time of command is 10 seconds. This command is valid after card activation sequence.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
 - BYTE AtiH[in] : Anti tearing integer H.
 - BYTE AtiM[in] : Anti tearing integer M.
 - BYTE AtiL[in] : Anti tearing integer L
$$ATI = Ati[H] \times 65536 + Ati[M] \times 256 + Ati[L]$$
$$\text{Anti-tearing time} = (ATI + 1) \times 100Us$$
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition .

3.13 DE_RFAntiTearing

int WINAPI DE_RFAntiTearing(int nPort, BYTE AtiH, BYTE AtiM, BYTE AtiL)

- This Function is for deactivate contactless card after a time defined in command data. The countdown starts after execute this function. Maximum execution time of command is 10 seconds. This command is valid after card activation sequence.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
 - BYTE AtiH[in] : Anti tearing integer H.
 - BYTE AtiM[in] : Anti tearing integer M.
 - BYTE AtiL[in] : Anti tearing integer L
$$ATI = Ati[H] \times 65536 + Ati[M] \times 256 + Ati[L]$$
$$\text{Anti-tearing time} = (ATI + 1) \times 100Us$$
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition .

4 ISO 18092 (FeliCa, NFC) API

4.1 DEC_Transparent

**int DEC_Transparent(int nPort, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes,
BYTE tout = 0xB0)**

- This function is for transparent communication with TYPE C card (FeliCa/ NFC) .
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in]: Command data.
 - LPINT outlen[out]: Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
 - BYTE tout[in] = 0xB0 : Time out value. Default time out value is 0xB0(300 ms)
- Return value

Success : DE_OK.

Failure : An error code. For more information, see Response code definition .

5 ISO 14443 TYPE B API

5.1 DEB_Transparent

int DEB_Transparent(int nPort, BYTE datalen, LPBYTE data, BYTE TOUT, LPINT outlen, LPBYTE lpRes)

- This function is for transparent communication with ISO 14443 TYPE B Card, CRC is excluded at command/response data, it is calculated from device. Device add CRC and send to Cards as the command data, and it deduct CRC and send to Host as response data
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - BYTE TOUT[in] : Time out value.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

5.2 DEB_TransparentCRC

int DEB_TransparentCRC(int nPort, BYTE datalen, LPBYTE data, LPBYTE crc, BYTE TOUT, LPINT outlen, LPBYTE lpRes)

- This function is for transparent communication with ISO 14443 TYPE B card, CRC is included at command/ response data. Device send receiving data to cards.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - LPBYTE crc[in] : 2 byte CRC.
 - BYTE TOUT[in] : Time out value.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6 ISO 14443 TYPE A API

6.1 DEA_Reset

int DEA_Reset(int nPort, LPBYTE lpdelay)

- This function deactivate(OFF) and activate(ON) RF field during the delay time.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE lpdelay[in] : Reset duration[LSB First]. Its length is 2. For example, {0x64, 0x00} means 100milli-second.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.2 DEA_Idle_Req

int DEA_Idle_Req(int nPort, LPINT outlen, LPBYTE lpRes)

- This function sends Idle Request command to Type A card. Halted cards don't send a response.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.3 DEA_Wakeup_Req

int DEA_Wakeup_Req(int nPort, LPINT outlen, LPBYTE lpRes)

- This function sends Wakeup Request command to Type A card. Halted cards become normal and send a response for this command
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPINT outlen[out] : Length of LPBYTE lpRes.

- LPBYTE lpRes[out] : Response data.

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.4 DEA_Anticoll

int DEA_Anticoll(int nPort, LPINT outlen, LPBYTE lpRes)

- This function sends Anticollision command to Type A card.(Only Cascade Level 1)

- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)

- LPINT outlen[out] : Length of LPBYTE lpRes.

- LPBYTE lpRes[out] : Response data.

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.5 DEA_Select

int DEA_Select(int nPort, LPBYTE uid, LPINT outlen, LPBYTE lpRes)

- This function send Select command to Type A card.

- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)

- LPBYTE uid[in] : 4 bytes UID.

- LPINT outlen[out] : Length of LPBYTE lpRes.

- LPBYTE lpRes[out] : Response data.

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.6 DEA_Auth

int DEA_Auth(int nPort, BYTE mode, BYTE keyno, BYTE blockno)

- This function sends Authentication command to selected Mifare type cards. For this process execute DEA_Loadkey command and save the key into the device is necessary,. Use DEA_Authkey command is recommendable instead it.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE mode[in] : 0x00 = Authentication with A key.
0x04 = Authentication with B key.
 - INT keyno[in] : Saved Key number(0~15)
 - BYTE blockno[in] : Block number to athenticate.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.7 DEA_Halt

int DEA_Halt(int nPort)

- This function sends HALT command. Halted cards don't send a response except Wakeup REQA.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.8 DEA_Read

int DEA_Read(int nPort, BYTE blockno, LPINT outlen, LPBYTE lpRes)

- This function reads one block(16Byte)which finished authenticated of the selected Mifare card.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.9 int DEA_Write

int DEA_Write(int nPort, BYTE blockno, int datalen, LPBYTE data)

- This function writes 16bytes date into the selected one block which finished authentocation of selected Mifare card.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
 - LPINT datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Response data.

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.10 DEA_Increment

int DEA_Increment(int nPort, BYTE blockno, LPBYTE value)

- This function reads data from authenticated value block, checks the data structure, increase the transmitted value and store the result into the card's internal register..
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
 - LPBYTE value[in] : increasing value(4 bytes).

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.11 DEA_Decrement

int DEA_Decrement(int nPort, BYTE blockno, LPBYTE value)

- This function reads data from authenticated value block, checks the data structure, decrease the transmitted value and store the result into the card's internal register..
- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- BYTE blockno[in] : Block number.
- LPBYTE value[in] : Decreasing value(4 bytes).
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.12 DEA_Inc_Transfer

int DEA_Inc_Transfer(int nPort, BYTE blockno, LPBYTE value, BYTE trblockno)

- This function reads data from authenticated value block, checks the data structure, increase the transmitted value and store the result into the card's selected block.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
 - LPBYTE value[in] : increasing value(4 bytes).
 - BYTE trblockno[in] : Block number to save(transfer) increased value
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.13 DEA_Dec_Transfer

int DEA_Dec_Transfer(int nPort, BYTE blockno, LPBYTE value, BYTE trblockno)

- This function reads data from authenticated value block, checks the data structure, decrease the transmitted value and store the result into the card's selected block.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
 - LPBYTE value[in] : Decreasing value(4 bytes).
 - BYTE trblockno[in] : Block number to save(transfer) decreased value
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.14 DEA_Restore

int DEA_Restore(int nPort, BYTE blockno)

- This function reads the value block, checks the data structure and stores the result into the card's internal register..
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.15 DEA_Transfer

int DEA_Transfer(int nPort, BYTE blockno)

- This function transfers the contents of the internal register to the card's selected block. The sector must be authenticated for this operation. The transfer function can be called directly after Increment, Decrement or Restore.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.16 DEA_Loadkey

int DEA_Loadkey(int nPort, BYTE mode, BYTE keyno, LPBYTE keydata)

- This function writes a master key into internal memory(SRAM) of reader and puts the master key buffer inside RF control chip. Reader can store 16 keys in internal memory for authentication procedure of accessed card. When power off, all keys are deleted. This command is used for authentication.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE mode[in] : 0x00 = Loading A key.

0x04 = Loading B key.

- BYTE keyno[in] : key number or index(0~15) for authentication.
- LPBYTE keydata[in] : Key data to be store into internal memory.(6 bytes)

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.17 DEA_Authkey

int DEA_Authkey(int nPort, BYTE mode, LPBYTE keydata, BYTE blockno)

- This function writes a secret key directly into the key buffer inside of RF control chip and authenticates selected sector with submitted key.

- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- BYTE mode[in] : 0x00 = Loading A key.

0x04= Loading B key.

- LPBYTE keydata[in] : Key data to be store into internal memory.(6 bytes)
- BYTE blockno[in] : The block number for authentication.

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.18 DEA_Req_Auth

int DEA_Req_Auth(int nPort, BYTE requestmode, BYTE authmode, BYTE keyno, BYTE blockno, LPINT outlen, LPBYTE lpRes)

- This function performs Request, Anti-collision, Authentication operations at once and receives the selected card's UID. This command use key data stored into internal memory. Before using this command, the DEA_Loadkey function should be executed.

- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- BYTE requestmode[in] : 0x00 = Request Idle.

0x01 = Request All.

- BYTE authmode[in] : 0x00 = Authentication with A key
- 0x04= Authentication with B key.

- LPBYTE keyno[in] : The key number or index(0~15) for authentication.
 - BYTE blockno[in] : The block number for authentication.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value
- Success: DE_OK.
- Failure : An error code. For more information, see Response code definition.

6.19 DEA_Req_Authkey

int DEA_Req_Authkey(int nPort, BYTE requestmode, BYTE authmode, BYTE blockno, LPBYTE keydata, LPINT outlen, LPBYTE lpRes)

- This function command performs Request, Anti-collision, Authentication operations with Key at once and returns the selected card UID. This command use submitted key for authentication
 - Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE requestmode[in] : 0x00 = Request Idle.
0x01 = Request All.
 - BYTE authmode[in] : 0x00 = Authentication with A key
0x04= Authentication with B key.
 - BYTE blockno[in] : The block number for authentication.
 - LPBYTE keydata[out] : Key data for authentication.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - Return value
- Success: DE_OK.
- Failure : An error code. For more information, see Response code definition.

6.20 DEA_Inc_Transfer2

Int DEA_Inc_Transfer2(int nPort, BYTE blockno, LPBYTE value, BYTE trblockno)

- This function reads the accessed value block, checks the data structure, increases the transmitted value, stores the result in the card's selected block and verify the result is correct

- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
 - LPBYTE value[in] : increasing value(4 bytes).
 - BYTE trblockno[in] : Block number to save the increased value
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.21 DEA_Dec_Transfer2

Int DEA_Dec_Transfer2(int nPort, BYTE blockno, LPBYTE value, BYTE trblockno)

- This function reads the accessed value block, checks the data structure, decreases the transmitted value, stores the result in the card's selected block and verifies the result is correct.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in] : Block number.
 - LPBYTE value[in] : Decreasing value(4 bytes).
 - BYTE trblockno[in] : Block number to save the decreased value
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.22 DEA_Req_AuthRead

int DEA_Req_AuthRead(int nPort, BYTE requestmode, BYTE authmode, BYTE keyno, BYTE blockno, LPINT outlen, LPBYTE lpRes)

- This function performs Request, Anti-collision, Authentication, read operations at once and receives the selected card UID & block data. Before using this command, the DEA_Loadkey function should be executed.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE requestmode[in] : 0x00 = Request Idle.
0x01 = Request All.

- BYTE authmode[in] : 0x00 = Authentication with A key.
0x04 = Authentication with B key.
- BYTE keyno[in] : Key number or index(0~15) for authentication.
- BYTE blockno[in]: Block number.
- LPINT outlen[out] : Length of LPBYTE lpRes.
- LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.23 DEA_Req_AuthkeyRead

**int DEA_Req_AuthkeyRead(int nPort, BYTE requestmode, BYTE authmode,
BYTE blockno, LPBYTE keydata, LPINT outlen, LPBYTE lpRes)**

- This function performs Request, Anti-collision, Authentication, read operations with Key at once and receives the selected card UID & block data of accessed block.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE requestmode[in] : 0x00 = Request Idle.
0x01 = Request All.
 - BYTE authmode[in] : 0x00 = Authentication with A key.
0x04 = Authentication with B key.
 - BYTE blockno[in]: Block number.
 - LPBYTE keydata[in] : Key data for authentication of selected block.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.24 DEA_Req_AuthWrite

**int DEA_Req_AuthWrite(int nPort, BYTE requestmode, BYTE authmode, BYTE keyno,
BYTE blockno, LPBYTE data, LPINT outlen, LPBYTE lpRes)**

- This function performs Request, Anti-collision, Authentication, Write operations at once and receives the selected card UID. Before using this command, the DEA_Loadkey function should be executed.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE requestmode[in] : 0x00 = Request Idle.
0x01 = Request All.
 - BYTE authmode[in] : 0x00 = Authentication with A key.
0x04 = Authentication with B key.
 - BYTE keyno[in] : Key number or index(0~15) for authentication.
 - BYTE blockno[in]: Block number.
 - LPBYTE data[in] : Data to write in selected block.(16 bytes)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.25 DEA_Req_AuthkeyWrite

**int DEA_Req_AuthkeyWrite(int nPort, BYTE requestmode, BYTE authmode,
BYTE blockno, LPBYTE keydata, LPBYTE data, LPINT outlen, LPBYTE lpRes)**

- This function performs Request, Anti-collision, Authentication, Write operations at once and receives the selected card UID. This command use summited key for authentication
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE requestmode[in] : 0x00 = Request Idle.
0x01 = Request All.
 - BYTE authmode[in] : 0x00 = Authentication with A key.
0x04 = Authentication with B key.
 - BYTE blockno[in]: Block number.
 - LPBYTE keydata[in] : Key data for authentication of selected block.
 - LPBYTE data[in] : Data to write in the selected block.

- LPINT outlen[out] : Length of LPBYTE lpRes.
- LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.26 DEA_Req_Select

int DEA_Req_Select(int nPort, BYTE requestmode, LPINT outlen, LPBYTE lpRes)

- This function performs Request, Anti-collision, Select operations at once and receives the UID of selected card. It supported cascade level 1 only.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE requestmode[in] : 0x00 = Request Idle.
0x01 = Request All.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.27 DEA_UltraM_Write

int DEA_UltraM_Write(int nPort, BYTE address, LPBYTE data)

- This function writes 4 bytes data to the selected address of Mifare ultra light card.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE address[in] : Address of Mifare ultra light card.
 - LPBYTE data[in]: Data to be written into the selected address(4 bytes).
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.28 DEA_AntiSelLevel

int DEA_AntiSelLevel(int nPort, LPINT outlen, LPBYTE lpRes)

- This function performs Anti-collision and Select command at once and receives UID. It supports Cascade level 1,2,3 . As a response it receives SAK & UID (4/7/10 bytes) Before using this command, Request function should be executed.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.(SAK +UID)
- Return value
Success: DE_OK
Failure : An error code. For more information, see Response code definition.

6.29 DEA_AnticollLevel

int DEA_AnticollLevel(int nPort, BYTE cmd, BYTE bitcnt, LPINT outlen, LPBYTE lpRes)

- This function performs Anti-collision by all Cascade level, bit mode level in order. As a response it receives UID (1 byte ID + 1 byte LRC) for each Cascade level. Refer to ISO 14443-3 specification for detail sequence of Anti-collision process.
- Parameter
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE cmd[in] : 0x93 = Cascade Level 1 (Equal or More than single UID).
0x95 = Cascade Level 2 (Equal or More than Double UID).
0x97 = Cascade Level 3 (Triple UID).
 - BYTE bitcnt[in] : Bit count.
 - BYTE uidlen[in] : Transferred UID number
 - LPBYTE uid[in] : UID data (Max 4 bytes)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.30 DEA_SelectLevel

int DEA_SelectLevel(int nPort, BYTE cmd, LPBYTE Uid, LPINT outlen, LPBYTE lpRes)

- This function performs Select sequence using 4bytes UID which got from Anti-collision.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE cmd[in] : 0x93 = Cascade Level 1 (Equal or More than single UID).
0x95 = Cascade Level 2 (Equal or More than Double UID).
0x97 = Cascade Level 3 (Triple UID).
 - LPBYTE Uid[in] : UID.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.31 DEA_DeviceInfo

int DEA_DeviceInfo(int nPort, LPINT outlen, LPBYTE lpRes)

- This function reads Product Serial Number(4bytes) of RF control chip from the reader .
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

6.32 DEA_Transparent

int DEA_Transparent(int nPort, BYTE datalen, LPBYTE data, BYTE TOUT, LPINT outlen, LPBYTE lpRes)

- This function sends the input data directly to cards and get response. Command data format should follow the ISO 14443 type A. CRC is calculated inside the device for sending Command and compared when it get the response.

- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - BYTE TOUT[in] : Time out value.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.33 DEA_TransparentCRC

**int DEA_TransparentCRC(int nPort, BYTE datalen, LPBYTE data, LPBYTE crc,
BYTE TOUT, LPINT outlen, LPBYTE lpRes)**

- This function sends the input data directly to cards and get response. Command data format should follow the ISO 14443 type A. User must calculate CRC and input it to command data. Response data includes CRC.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - LPBYTE crc[in] : 2 byte CRC.
 - BYTE TOUT[in] : Time out value.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.34 DEA_BitMode

**int DEA_BitMode(int nPort, BYTE datalen, BYTE TxByteNo, BYTE TxBitNo,
LPBYTE data, LPINT outlen, LPBYTE lpRes);**

- This function sends user data to the card as bit mode. It can send data to be transmitted in a bit unit. If collision occurred during receiving, byte count number and bit position number in a collision byte is included in response data. The received data always aligns from bit position 0.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - BYTE TxByteNo[in] : Byte count number.
 - BYTE TxBitNo[in] : Bit count number.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.For more information, see DE-620 Protocol spec(3.2.33 _DEA_BITMODE).
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.35 DEA_BitModeAnti

**int DEA_BitModeAnti(int nPort, BYTE datalen, BYTE TxByteNo, BYTE TxBitNo,
LPBYTE data, LPINT outlen, LPBYTE lpRes);**

- This function is for tests anti-collision function of the card as bit mode. This command is very useful for test of anti-collision performance step by step. It sends data as bit unit. If collision occurred during receiving data, byte count number and bit position number will be included in the response data. The received data always aligns to the position which defined in TxBitNo.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - BYTE TxByteNo[in] : Byte count number.
 - BYTE TxBitNo[in] : Bit count number.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data. (Byte count + Bit count + Response data)For more information, see DE-620 Special Protocol spec(3.2.34 _DEA_BITMODEANTI).
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.36 DEA_BitMode2

**int DEA_BitMode2(int nPort, BYTE datalen, BYTE TxByteNo, BYTE TxBitNo,
LPBYTE data, LPINT outlen, LPBYTE lpRes);**

- This function is same as DEA_Bitmode except it doesn't parity check of response data
- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- int datalen[in] : Length of LPBYTE data.
- LPBYTE data[in] : Command data.
- BYTE TxByteNo[in] : Byte count number.
- BYTE TxBitNo[in] : Bit count number.
- LPINT outlen[out] : Length of LPBYTE lpRes.
- LPBYTE lpRes[out] : Response data. (Byte count + Bit count + reseponse data)

For more information, see DE-620 special Protocol spec(3.2.35 _DEA_BITMODE2).

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

6.37 DE_MifarePlus_Activation

**int DE_MifarePlus_Activation(int nPort, LPBYTE Master_Key, LPBYTE Conf_Key,
LPBYTE L2sw_Key, LPBYTE L3sw_Key,
LPBYTE SAK, LPBYTE UID, LPINT UID_Length);**

- This command is used to active MifarePlus Card to MifarePlus Type1 Card.
- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- LPBYTE Master_Key[in] : Card Master Key(16 Bytes).
- LPBYTE Conf_Key[in] : Card Configuration Key(16 Bytes).
- LPBYTE L2sw_Key[in] : Level2 Switch Key(16 Bytes).
- LPBYTE L3sw_Key[in] : Level3 Switch Key(16 Bytes).
- LPBYTE SAK[out] : SAK data.
- LPBYTE UID[out] : UID data.

- LPINT UID_Length[out] : Length of UID.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

7 ISO14443 TYPE A/B Common API

7.1 DE_APDU

int DE_APDU(int nPort, BYTE datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes)

- It sends and receive data as APDU format defined by ISO7816. Device change it to ISO14443 protocol and send to card, so User can easily use this function without knowing of ISO14443 protocol.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : APDU to send to card.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

7.2 DE_FindCard

**int DE_FindCard(int nPort, BYTE baud, BYTE cid, BYTE nad, BYTE option,
LPINT outlen, LPBYTE lpRes)**

- This function detects card in the RF field and receive response data. User can set communication speed/CID/NAD/Card type and send command to device, device change it as ISO14443 protocol to communicate to card.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int baud[in] : 0x00 = 106 kbps.
0x01 = 212 kbps.
0x02 = 424 kbps.
0x03 = 848 kbps
 - BYTE cid[in] : CID.
 - BYTE nad[in] : NAD
 - BYTE option[in] : 'A'(0x41): detects only A-type card, 'B'(0x42): detect only B-type card, other: detect all A and B type card (normally use 0x00).

- LPINT outlen[out] : Length of LPBYTE lpRes.
- LPBYTE lpRes[out]: Response data. (Card type, Communication speed, Response data)
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

7.3 DEAB_RwWrite

int DEAB_RwWrite(int nPort, BYTE Data)

- This function is for writing reader's serial number to reader's EEPROM memory. It doesn't deleted when power off
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE Data[in]: 1 Byte Data. (Device/Reader serial number)
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

7.4 DEAB_RwRead

int DEAB_Rw_Read(int nPort, LPBYTE lpRes)

- This function is for reading reader's serial number from reader's EEPROM memory.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE lpRes[out]: Response data..(Device/Reader serial number)
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

8 ISO15693 API

8.1 DED_Inventory

int DED_Inventory(int nPort, BYTE Flag, LPINT outlen, LPBYTE lpRes)

- This function sends the inventory, Anticollision command and receives Cards number and 61bit UID in RF field. In case of next command is Select, it is selected last checked card. Earlier checked card set as Quiet mode.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE Flag[in] : Flag(0x01 or 0x10).
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK

Failure : An error code. For more information, see Response code definition.

8.2 DED_Select

int DED_Select(int nPort, LPINT outlen, LPBYTE lpRes)

- This function selects the last checked cards from 7.1 command.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

8.3 DED_Read

int DED_Read(int nPort, BYTE blockno, LPINT outlen, LPBYTE lpRes)

- This function reads specified block of selected card.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)

- BYTE blockno[in]: Block number to read.
- LPINT outlen[out] : Length of LPBYTE lpRes.
- LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

8.4 DED_Write

int DED_Write(int nPort, BYTE blockno, LPBYTE data, LPINT outlen, LPBYTE lpRes)

- This function writes 4 bytes data into specified block of selected card.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE blockno[in]: Block number to read.
 - LPBYTE data[in] : 4 bytes data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

8.5 DED_Transparent

int DED_Transparent(int nPort, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes)

- It sends data which received from Host to Card and get response data as ISO15693 format. CRC is calculatated by reader.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE datalen[in]: Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

8.6 DED_Eof

int DED_Eof(int nPort, LPINT outlen, LPBYTE lpRes)

- When there are multiple cards in RF fields, it sends Change slot command to communicate with other cards besides current one. Next command will be received by defined slot card
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9 ISO7816 API

9.1 DE_IC_PowerOn

int DE_IC_PowerOn(int nPort, BYTE slotno, LPINT outlen, LPBYTE lpRes)

- This function activates selected card slot.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : Contact slot number.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.2 DE_IC_PPS

**int DE_IC_PPS(int nPort, BYTE slotno, int ppslen, LPBYTE ppsdata,
LPINT outlen, LPBYTE lpRes)**

- This function sends PPS command to selected card slot
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : Contact slot number.
 - Int ppslen[int] : Length of LPBYTE ppsdata.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.3 DE_IC_Case1

**int DE_IC_Case1(int nPort, BYTE slotno, int datalen, LPBYTE data, LPINT outlen,
LPBYTE lpRes)**

- This function sends 5 bytes APDU and receives 2 bytes(status words). This function support case 1 APDU in ISO/IEC 7816-3.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : Contact slot number.
 - Int datalen[int] : Length of LPBYTE data.
 - LPBYTE data[in]: APDU data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.4 DE_IC_Case2

int DE_IC_Case2(int nPort, BYTE slotno, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes)

- This function sends APDU(5 bytes) and receives N bytes(data and status words). This function support case 2 APDU in ISO/IEC 7816-3.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : Contact slot number.
 - Int datalen[int] : Length of LPBYTE data.
 - LPBYTE data[in]: APDU data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.5 DE_IC_Case3

int DE_IC_Case3(int nPort, BYTE slotno, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes)

- This function sends 5 bytes APDU + N bytes data and receives 2 bytes (status words). This function support case 3 APDU in ISO/IEC 7816-3.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : Contact slot number.
 - Int datalen[int] : Length of LPBYTE data.
 - LPBYTE data[in]: APDU data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.6 DE_IC_Case4

int DE_IC_Case4(int nPort, BYTE slotno, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes)

- This function sends 5 bytes APDU+ N bytes data+1 byte Le(expected length of response data) and receives N bytes(data and status words). This function support case 4 APDU in ISO/IEC 7816-3.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : Contact slot number.
 - Int datalen[int] : Length of LPBYTE data.
 - LPBYTE data[in]: APDU data.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.7 DE_IC_PowerOff

int DE_IC_PowerOff(int nPort, BYTE slotno)

- This function deactivates selected card slot (power off).
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : Contact slot number.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.8 DE_T1Bypass

int DE_T1Bypass(int nPort, int apdulen, BYTE slotno, BYTE nad, BYTE pcb, BYTE lenth, LPBYTE apdu, BYTE lrc, LPINT outlen, LPBYTE lpRes)

- This function bypasses data to the selected card slot and receives data as T1 format.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE apdulen[in] : Length of LPBYTE apdu.
 - BYTE slotno[int] : Contact slot number.
 - BYTE nad[in]: NAD.
 - BYTE pcb[in] : PCB.
 - BYTE lenth[in]: LEN.
 - LPBYTE apdu[in] : APDU.
 - BYTE lrc[in] : LRC.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.9 DE_IC_Speed

int DE_IC_Speed(int nPort, BYTE slotno, BYTE fidi)

- This function changes the communication speed of the card slot.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : Contact slot number.

- BYTE fidi[in] : FI and DI value of ISO 7816-3.

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

9.10 DE_CARD_APDU

**int DE_CARD_APDU(int nPort, BYTE slotno, int datalen, LPBYTE data, LPINT outlen,
LPBYTE lpRes)**

- This function sends data to the selected card slot and receives data as T1 format.

- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
- BYTE slotno[in] : Contact slot number.
- Int datalen[in]: Length of LPBYTE data.
- LPBYTE data[in] : APDU+data.
- LPINT outlen[out] : Length of LPBYTE lpRes.
- LPBYTE lpRes[out] : Response data.

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

10 FeliCa Commands

10.1 DEC_Transparent

int DEC_Transparent(int nPort, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes, BYTE tout = 0xB0);//default timeout:300ms

- This function sends the input data directly to cards and get response. Command data format should follow the Felica Card command specification.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Command data.
 - BYTE TOUT[in] : Time out value.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - BYTE tout[in] : default time out setting is 300ms. it only used for serial communication.
 - Return value
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

10.2 DEC_Polling_NoENC

int DEC_Polling_NoENC(int nPort, LPBYTE systemcode, BYTE requestsyscode, BYTE timeslot, LPINT outlen, LPBYTE lpRes, BYTE tout = 0xB0);//default timeout:300ms

- This function use for sending polling command to FeliCa Card.
- This function use [Polling command] for a non-encrypted Service in a FeliCa card.
- You can get Manufacture ID (IDm) and Manufacture Parameter(PMm) of Felica Card
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE systemcode[in] : system code.
 - LPBYTE requestsyscode[in] :
 - ◆ 01h : system code is requested
 - ◆ Others : system code is not requested.
 - BYTE timeslot[in] : Time slot

◆ 00h/01h/03h/07h/0fh.

- LPINT outlen[out] : Length of LPBYTE lpRes.
- LPBYTE lpRes[out] : Response data.
- BYTE tout[in] : default time out setting is 300ms. it only used for serial communication.
- Return value

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

10.3 DEC_Read_NoENC

int DEC_Read_NoENC(int nPort, LPBYTE IDm, LPBYTE servicecode, BYTE block, LPINT outlen, LPBYTE lpRes, BYTE tout = 0xB0);//default timeout:300ms

- This function use for reading data of a FeliCa Card.
- This function use [Read command] for a non-encrypted Service in a FeliCa card.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE IDm[in] : IDm(Manufacture ID).
 - LPBYTE servicecode[in] : service code to read
 - BYTE block[in] : block number to write
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - BYTE tout[in] : default time out setting is 300ms. it only used for serial communication.
 - Return value

- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

10.4 DEC_Write_NoENC

int DEC_Write_NoENC(int nPort, LPBYTE IDm, LPBYTE servicecode, BYTE block, LPBYTE blockdata, LPINT outlen, LPBYTE lpRes, BYTE tout = 0xB0);//default timeout:300ms

- This function use for writing data to the FeliCa Card.
- This function use [Write command] for a non-encrypted Service in a FeliCa card
- Parameters

- int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE IDm[in] : IDm(Manufacture ID).
 - LPBYTE servicecode[in] : service code to write
 - BYTE block[in] : block number to write
 - LPBYTE blockdata[in] : data to write
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - BYTE tout[in] : default time out setting is 300ms. it only used for serial communication.
 - Return value
- Return value
- Success: DE_OK.
- Failure : An error code. For more information, see Response code definition.

10.5 DEC_ControllernRWAAuth

int DEC_ControllernRWAAuth(int nPort, BYTE slotno, BOOL bUseEncCmd, LPBYTE key, LPBYTE CBC)

- This function use for authentication RC-S251 SAM.
 - This function executes authentication commands like below order
 - [Set RWSAM Mode command]
 - [Attention command],
 - [Authentication1 command]
 - [Authentication2 command]
 - Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE slotno[in] : number of SAM slot
 - BOOL bUseEncCmd[in] :
 - ◆ TRUE : Use Encrypted command (Make a command use key and CBC)
 - ◆ FALSE : Use plain command
 - LPBYTE key[in] : key data to encrypt command
 - LPBYTE CBC[in] : cbc data to encrypt command
 - Return value
- Success: DE_OK.
- Failure : An error code. For more information, see Response code definition.

10.6 DEC_FeliCaPolling

int DEC_FeliCaPolling(int nPort, BOOL bUseEncCmd, LPBYTE syscode, BYTE timeslot, LPINT outlen, LPBYTE lpRes)

- This function use for sending polling command to FeliCa Card.
- This function use [Polling command] for a encrypted Service in a FeliCa card.
- This function uses RC-S251 SAM to make a secured command.
- You can get Manufacture ID (IDm) and Manufacture Parameter(PMm) of Felica Card
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BOOL bUseEndCmd[in] :
 - ◆ TRUE : Use Encrypted command (Make a command use key and CBC)
 - ◆ FALSE : Use plain command
 - LPBYTE sysode[in] : system code..
 - BYTE timeslot[in] : Time slot
 - ◆ 00h/01h/03h/07h/0fh.
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

10.7 DEC_MutualAuth

int DEC_MutualAuth(int nPort,BOOL nuseTdes,LPBYTE pGroupKey,LPBYTE pUserKey,LPBYTE AreaCode,LPBYTE ServiceCode)

- This function use for mutual authentication of Felica card and RS-S251 SAM with Group Service Key and User Service Key.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BOOL nuseTdes[in] :
 - ◆ TRUE : Use Encrypted command
 - ◆ FALSE : Use plain command
 - LPBYTE pGroupKey[in] : Group Service Key.

- LPBYTE pUserKey[in] : User Service Key
- LPBYTE AreaCode[in] : Area code
- LPBYTE ServiceCode[in] : Service code
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

10.8 DEC_MutualAuth_RWSAM

int DEC_MutualAuth_RWSAM(int nPort,BOOL nuseTdes,LPBYTE Systemcode,LPBYTE GSKcode,LPBYTE GSKversion,LPBYTE USKcode,LPBYTE USKversion)

- This function use for mutual authentication of Felica card and RS-S251 SAM with Group Service Key code and User Service Key code.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BOOL nuseTdes[in] :
 - ◆ TRUE : Use Encryped command
 - ◆ FALSE : Use plain command
 - LPBYTE Systemcode[in] : System code
 - LPBYTE GSKcode[in] : Group Service Key code.
 - LPBYTE GSKversion[in] : Group Service Key version
 - LPBYTE USKcode[in] : User Service Key code.
 - LPBYTE USKversion[in] : User Service Key version.
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

10.9 DEC_ReadCardwithENC

int DEC_ReadCardwithENC(int nPort,LPBYTE pGroupKey,LPBYTE pUserKey,LPBYTE AreaCode,LPBYTE ServiceCode,BYTE BolockCnt, BYTE byteferblock,LPBYTE BlockList,BOOL nuseTdes, LPINT outlen, LPBYTE lpRes)

- This function use for mutual authentication and reading data of a FeliCa Card at one time.
- This function use [Read command] for a encrypted Service in a FeliCa card
- The block list uses 2 or 3 bytes per block to describe all necessary attributes.

- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE pGroupKey[in] : Group Service Key.
 - LPBYTE pUserKey[in] : User Service Key.
 - LPBYTE Areacode[in] : Area Code.
 - LPBYTE ServiceCode[in] : Service Code.
 - BYTE BlockCnt[in] : Count of block to read.
 - BYTE byteferblock[in]: Byte count per block of block list.
 - LPBYTE BlockList[in] : List of block to read.
 - BOOL nuseTdes[in] :
 - ◆ TRUE : Use Encryped command
 - ◆ FALSE : Use plain command
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - Return value
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

10.10 DEC_WriteCardwithENC

int DEC_WriteCardwithENC(int nPort,LPBYTE pData,LPBYTE pGroupKey,LPBYTE pUserKey,LPBYTE AreaCode,LPBYTE ServiceCode,BYTE BolockCnt, BYTE byteferblock, LPBYTE BlockList,BOOL nuseTdes)

- This function use for mutual authentication and writing data to the FeliCa Card at one time.
- This function use [Write command] for a encrypted Service in a FeliCa card
- The block list uses only 2 bytes per block to describe all necessary attributes.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE pData[in] : Data to write.
 - LPBYTE pGroupKey[in] : Group Service Key.
 - LPBYTE pUserKey[in] : User Service Key.
 - LPBYTE Areacode[in] : Area Code.
 - LPBYTE ServiceCode[in] : Service Code.

- BYTE BlockCnt[in] : Count of block to read.
- LPBYTE BlockList[in] : List of block to read.
- BYTE byteferblock[in]: Byte count per block of block list.
- BOOL nuseTdes[in] :
 - ◆ TRUE : Use Encrypted command
 - ◆ FALSE : Use plain command

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

10.11 DEC_ReadCardwithENC_NoMA

int DEC_ReadCardwithENC_NoMA(int nPort,LPBYTE AreaCode,LPBYTE ServiceCode, BYTE BolockCnt, BYTE byteferblock, LPBYTE BlockList,BOOL nuseTdes, LPINT outlen, LPBYTE lpRes)

- This function use for only reading data of a FeliCa Card.
- This function use [Read command] for a encrypted Service in a FeliCa card.
- You should use this function after you execute DEC_MutualAuth function or DEC_MutualAuth_RWSAM function
- The block list uses only 2 bytes per block to describe all necessary attributes.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE AreaCode[in] : Area Code.
 - LPBYTE ServiceCode[in] : Service Code.
 - BYTE BlockCnt[in] : Count of block to read.
 - BYTE byteferblock[in]: Byte count per block of block list.
 - LPBYTE BlockList[in] : List of block to read.
 - BOOL nuseTdes[in] :
 - ◆ TRUE : Use Encrypted command
 - ◆ FALSE : Use plain command
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
- Return value

Success: DE_OK.

Failure : An error code. For more information, see Response code definition.

10.12 DEC_WriteCardwithENC_NoMA

int DEC_WriteCardwithENC_NoMA(int nPort,LPBYTE pData,LPBYTE AreaCode,LPBYTE ServiceCode, BYTE BolockCnt, BYTE byteferblock, LPBYTE BlockList,BOOL nuseTdes)

- This function use for only writing data to the FeliCa Card.
- This function use [Write command] for a encrypted Service in a FeliCa card.
- You should use this function after you execute DEC_MutualAuth function or DEC_MutualAuth_RWSAM function
- The block list uses only 2 bytes per block to describe all necessary attributes.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPBYTE pData[in] : Data to write.
 - LPBYTE Areacode[in] : Area Code.
 - LPBYTE ServiceCode[in] : Service Code.
 - BYTE BlockCnt[in] : Count of block to read.
 - BYTE byteferblock[in]: Byte count per block of block list.
 - LPBYTE BlockList[in] : List of block to read.
 - BOOL nuseTdes[in] :
 - ◆ TRUE : Use Encryped command
 - ◆ FALSE : Use plain commandReturn value
- Return value
Success: DE_OK.
Failure : An error code. For more information, see Response code definition.

10.13 DEC_RequestService

int DEC_RequestService(int nPort,BOOL nuseTdes, BYTE nAreaORService, LPBYTE pCodeList, LPINT outlen, LPBYTE lpRes)

- This function use for checking area code and service code.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BOOL nuseTdes[in] :
 - ◆ TRUE : Use Encryped command
 - ◆ FALSE : Use plain commandReturn value

- BYTE nAreaORService[in] : Count of area or service code to check.
 - LPBYTE pCodeList[in] : List of code to check
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out] : Response data.
 - ◆ Data
 - IDtr[8] + No. of Area/Service "n"[1] + Area/Service key version list
 - Return value
- Success: DE_OK.
- Failure : An error code. For more information, see Response code definition.

11 Fingerprint Command

11.1 DE_FingerGetImageStart

int DE_FingerGetImageStart(int nPort, BYTE* Imagepos)

- This function use for getting fingerprint image from the DE-600F Reader. When the reader receives this command, It makes fingerprint image and saves in the memory.
- Parameters
 - int nPort[in] : Device number. (See nPort definition in DE_InitPort funtion)
 - BYTE* imagepos[out] : position of your fingerprint.
 - ◆ PLACE_FINGER 0xFF (waiting finger)
 - ◆ LIFT_FINGER 0xFE (lift your finger from the reader)
 - ◆ PLACE_OK 0 (position of finger is correct)
 - ◆ UP_MOVE 1 (move your finger to up side)
 - ◆ DOWN_MOVE 2 (move your finger to down side)
 - ◆ LEFT_MOVE 4 (move your finger to left side)
 - ◆ RIGHT_MOVE 8 (move your finger to right side)
 - ◆ DOWN_LEFT_MOVE 9 (move your finger to down and left side)
 - ◆ DOWN_RIGHT_MOVE 10 (move your finger to down and right side)
 - ◆ UP_LEFT_MOVE 11 (move your finger to up and left side)
 - ◆ UP_RIGHT_MOVE 12 (move your finger to up and right side)
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, see Response code definition (page 51).

11.2 DE_FingerGetImageData

int DE_FingerGetImageData(int nPort, BYTE imageflag, char* imagepath)

- This function use for getting fingerprint image data from the DE-600F Reader's memroy.
- Parameters
 - int nPort[in] : Device number. (See nPort definition in DE_InitPort funtion)
 - BYTE imageflag[in] : Flag of image size to get
 - ◆ 0 : 128*180 size bmp image

- ◆ 1 : 256 * 360 size bmp image
- Char* imagepath[in] : Path of bmp image to save
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, see Response code definition(page 51).

11.3 DE_FingerEnroll

int DE_FingerEnroll(int nPort,int* quality,int* processtime,BYTE* templatedata,int* templaten)

- This function use for getting fingerprint's template data.
- Parameters
 - int nPort[in] : Device number. (See nPort definition in DE_InitPort funtion)
 - int* quality[out] : Quality of fingerprint image
 - int* processtime[out] : Processing time of enroll fingerprint function.
 - BYTE* templatedata[out] : Template data of fingerprint.
 - int* templaten[out] : Size of template data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, see Response code definition(page 51).

11.4 DE_FingerVerify

int DE_FingerVerify(int nPort,int* quality,int* processtime)

- This function use for checking fingerprint.
 - If you use this function without using DE_FingerDownloadtemplate function, the reader compares fingerprint with the reader's own memroy data(The reader saves template data in the its own memory whenever you enroll fingerpirnt. The memory can save 5 template data. But I If you power off the reader, that template data is cleared.)
- Parameters
 - int nPort[in] : Device number. (See nPort definition in DE_InitPort funtion)
 - int* quality[out] : Quality of fingerprint image
 - int* processtime[out] : Processing time of verify fingerprint function.

- Return value
Success : DE_OK.
Failure : DE_ERROR

11.5 DE_FingerCalibrate

int DE_FingerCalibrate(int nPort)

- This function use for calibrating the fingerprint module.
- It takes some time. So you should not power off until get return value.
- Parameters
 - int nPort[in] : Device number. (See nPort definition in DE_InitPort funtion)
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, see Response code definition(page 51).

11.6 DE_FingerDownloadTemplate

**int DE_FingerDownloadTemplate(int nPort,BYTE index,BYTE* templatedata,int
templaten)**

- This function use for downloading fingerprint's template data to the reader.
- Parameters
 - int nPort[in] : Device number. (See nPort definition in DE_InitPort funtion)
 - int index[in] : Index of fingerprint template(0~4). You must send index in the order.
 - BYTE* templatedata[in] : Template data of fingerprint.
 - int templaten[in] : Size of template data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12 LLC Commands

12.1 LLC_MAC_Activation

int LLC_MAC_Activation(int nPort, BYTE NFCMode)

- This function can be used only when service access point (or service user) want to activate a MAC link between NFC devices according to LLCP specification.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE NFCMode[in] : Mode.
 - ◆ 0x41('A') = Active initiator
 - ◆ 0x50('P') = Passive initiator
 - ◆ 0x74('t') = Target
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.2 LLC_MAC_DeActivation

int LLC_MAC_DeActivation(int nPort, BYTE type)

- This function can be used only when service access point (or service user) want to deactivate a MAC link between NFC devices.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE type[in] : Type.
 - ◆ TYPE = 0x00, Deselect
 - ◆ TYPE = 0x01, Release
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.3 LLC_Connect_Request

int LLC_Connect_Request(int nPort)

- This function is used to send CONNECT PDU by local LLC as initiator to remote LLC after MAC activation. This function can be used only when service access point (or service user) wants local LLC as initiator to connect with remote LLC.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.4 LLC Connect Response

int LLC_Connect_Response(int nPort, BYTE sel, BYTE dir, BYTE Reason)

- This function is used to send CC PDU or DM PDU by local LLC as initiator for the response of CONNECT or DISC PDU.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE sel[in] : Send CC or DM .
 - ◆ SEL=0, Send CC(Connection Complete), Length = 0x0002
 - ◆ SEL=1, Send DM(Disconnected Mode), Length = 0x0004
 - BYTE dir[in] : Outbound or Inbound.
 - ◆ DIR = 0, Outbound (Outgoing link at local LLC to remote LLC)
 - ◆ DIR = 1, Inbound (Ingoing link at local LLC from remote LLC)
 - BYTE Reason[in] : Reason of link disconnection. Please refer to LLC specifications.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.5 LLC_Disconnect_Request

int LLC_Disconnect_Request(int nPort, BYTE dir)

- This function is used to send DISC PDU by LLC as initiator to terminate a data link connection or deactivate the LLCP Link.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE dir[in] : Outbound or Inbound.
 - ◆ DIR = 0, Outbound (Outgoing link at local LLC)
 - ◆ DIR = 1, Inbound (Ingoing link at local LLC)
 - ◆ DIR = 2, Intentional disconnect at a LLC
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.6 LLC_Send_Recieve_Ready

int LLC_Send_Recieve_Ready(int nPort, BYTE dir, LPINT outlen, LPBYTE lpRes)

- This function is used by an LLC to send RR PDU by LLC as initiator to acknowledge one received I PDU and indicate that the local LLC is able to receive subsequent I PDU.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE dir[in] : Outbound or Inbound.
 - ◆ DIR = 0, Outbound (Outgoing link at local LLC)
 - ◆ DIR = 1, Inbound (Ingoing link at local LLC)
 - LPINT outlen[in] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[in] : Response data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.7 LLC_Send_Symmetry

int LLC_Send_Symmetry(int nPort)

- This function is used to send SYMM PDU by LLC as initiator to indicate that no other PDUs are available for sending, to ensure symmetry.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.8 LLC_Send_Information

int LLC_Send_Information(int nPort, BYTE dir, int nData, LPBYTE pData)

- This function is used to transfer service data unit by LLC as initiator across a data link connection.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE dir[in] : Outbound or Inbound.
 - ◆ DIR = 0, Outbound (Outgoing link at local LLC)
 - ◆ DIR = 1, Inbound (Ingoing link at local LLC)
 - int nData[in] : Length of LPBYTE pData.
 - LPBYTE pData[in] : SDU, LLC PDU or NDEF message data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.9 LLC_Send_Text

int LLC_Send_Text(int nPort, BYTE type, int nData, LPBYTE pData)

- This function is batch command to send Text-type NDEF message to remote LLC configured as target when the local LLC is configured as initiator. All LLC-related commands are automatically executed in the reader.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE type[in] : Type.
 - ◆ TYPE = 0x00, Text-type, Normal Link Connection including MAC Activation & Deactivation
 - ◆ TYPE = 0x10, Text-type, Link Connection without MAC Activation
 - ◆ TYPE = 0x80, Text-type, Link Connection without MAC Deactivation
 - int nData[in] : Length of LPBYTE pData.
 - LPBYTE pData[in] : Text.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

12.10 LLC_Receive_Text

int LLC_Receive_Text(int nPort, BYTE pType, LPINT outlen, LPBYTE lpRes)

- This function is batch command to receive NDEF message from remote LLC configured as target when the local LLC is configured as initiator. All LLC-related commands are automatically executed in the reader.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE pType [in] : Type.
 - ◆ TYPE = 0x00, Normal Link Connection including MAC Activation & Deactivation
 - ◆ TYPE = 0x10, Link Connection without MAC Activation
 - ◆ TYPE = 0x80, Link Connection without MAC Deactivation
 - LPINT outlen [in] : Length of LPBYTE lpRes.
 - LPBYTE lpRes [in] : Response data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

13 ISO-18092 Commands (optional functions)

13.1 DE_NFC_Init_INITIATOR

int DE_NFC_Init_INITIATOR(int nPort, BYTE mode, BYTE speed, LPINT outlen, LPBYTE lpRes)

- This function sets the NFC device to initiator mode and performs RF Collision Avoidance.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
 - BYTE Mode[in] : Mode.
 - ◆ 0x41('A') = Active initiator
 - ◆ 0x50('P') = Passive initiator
 - BYTE speed[in] : Communication speed.
 - ◆ 00 : 106kbps
 - ◆ 01 : 212kbps
 - ◆ 02 : 424kbps
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

13.2 DE_NFC_Init_TARGET

int DE_NFC_Init_TARGET(int nPort, BYTE timeout, LPINT outlen, LPBYTE lpRes)

- This function sets the NFC device to target mode and the target will be answered automatically for the Polling and ATR_REQ command.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort function)
 - BYTE timeout[in] : Time-out value (Target will wait the data from Initiator by this value)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

13.3 DE_NFC_SendData

int DE_NFC_SendData(int nPort, BYTE mode, int datalen, LPBYTE data, LPINT outlen, LPBYTE lpRes)

- This function can be used only when the reader is configured as initiator or target.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE mode[in] : Mode.
 - ◆ 0 : Target Mode
 - ◆ 1 : Initiator Mode
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Data to be sent(Max. Length : 500 bytes).
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

13.4 DE_NFC_Stop

int DE_NFC_Stop(int nPort, BYTE code)

- This function is used to send Deselect/Release/Wakeup Request in initiator mode to the target and end the operation of target in abnormal cases of the target.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE code[in] : Code.
 - ◆ 0 : Initiator sends DSL_REQ.
 - ◆ 1 : Initiator sends RLS_REQ.

- ◆ 2 : It force Target to stay Initial_State of the device.

Target device can't receive the Initiator command any more.

- ◆ 3 : Initiator sends WUP_REQ used only in active mode communication.

- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

13.5 DE_NFC_GetTargetData

int DE_NFC_GetTargetData(int nPort, LPINT outlen, LPBYTE lpRes)

- This function can be used only when the reader is configured as target. After connecting with initiator, the reader as target receives data frame from the initiator and manages protocol management and error handling.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

13.6 DE_NFC_SetTargetData

int DE_NFC_SetTargetData(int nPort, int datalen, LPBYTE data)

- This function can be used only when the reader is configured as target. After connecting with initiator, the reader as target sends data frame received from host to the initiator and manages protocol management and error handling.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - int datalen[in] : Length of LPBYTE data.
 - LPBYTE data[in] : Data to be sent.

- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

13.7 DE_NFC_GetTargetState

int DE_NFC_GetTargetState(int nPort, LPINT outlen, LPBYTE lpRes)

- This function is used to know what the current state of the target is.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - LPINT outlen[out] : Length of LPBYTE lpRes.
 - LPBYTE lpRes[out]: Response data.
- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

13.8 DE_NFC_TAG_CMD

int DE_NFC_TAG_CMD(int nPort, BYTE TagType, BYTE TagCMD, int optdatalen, LPBYTE optdata, LPINT outlen, LPBYTE lpRes)

- This function is used to know what the current state of the target is.
- Parameters
 - int nPort[in] : Port number. (See nPort definition in DE_InitPort funtion)
 - BYTE TagType[in] : NFC Tag Type.
 - ◆ 0x10 = Type 1
 - ◆ 0x20 = Type 2
 - ◆ 0x30 = Type 3
 - ◆ 0x40 = Type 4
 - BYTE TagCMD : command code to be executed.
 - int optdatalen : Length of LPBYTE optdata.
 - LPBYTE optdata : Optional Data according to command format.

- LPINT outlen[out] : Length of LPBYTE lpRes.
- LPBYTE lpRes[out]: Response data.

- Command frame

For Type 1 Tag(Tag Type = 0x10)

- Read Identifier (RID), Tag CMD=0x00

CMD	Tag Type	Tag CMD
0x9E	0x10	0x00

- Read All, Tag CMD=0x01

Read all blocks from 0x00 to 0x0E in a static memory including Header ROM bytes
(Total 130bytes = 128bytes user data + 2bytes HR)

CMD	Tag Type	Tag CMD
0x9E	0x10	0x01

- Read a byte, Tag CMD=0x02

Read single byte in a static memory

CMD	Tag Type	Tag CMD	Data[2]	Data[3]
0x9E	0x10	0x02	Block Addr	Byte Offset

Data[2] : Block address (0x00 ~ 0x0E)

Data[3] : Byte offset (0x00 ~ 0x07)

- WriteE, Tag CMD=0x03

Write single byte with erase in a static memory

CMD	Tag Type	Tag CMD	Data[2]	Data[3]	Data[4]
0x9E	0x10	0x03	Block Addr	Byte Offset	DATA

Data[2] : Block address (0x00 ~ 0x0E)

Data[3] : Byte offset (0x00 ~ 0x07)

Data[4] : Data to be written

- WriteNE, Tag CMD=0x04

Write single byte without erase in a static memory

CMD	Tag Type	Tag CMD	Data[2]	Data[3]	Data[4]
0x9E	0x10	0x04	Block Addr	Byte Offset	DATA

Data[2] : Block address (0x00 ~ 0x0E)

Data[3] : Byte offset (0x00 ~ 0x07)

Data[4] : Data to be written

For Type 2 Tag (Tag Type = 0x20)

- Read Block, Tag CMD=0x00

CMD	Tag Type	Tag CMD	DATA[2]
0x9E	0x20	0x00	BN

Data[2] : Block Number to be accessed. 1 block consists of 4bytes but reading 1 block returns 4 block data (16bytes)

- Write Block, Tag CMD=0x01

CMD	Tag Type	Tag CMD	DATA[2]	Data[3] ... Data[6]
0x9E	0x20	0x01	BN	DATA (4bytes)

Data[2] : Block Number to be accessed. 1 block consists of 4bytes but reading 1 block returns 4 block data (16bytes)

Data[3] ... Data[6] : 4 bytes data to be written for the specified block.

- Sector Select, Tag CMD=0x02

CMD	Tag Type	Tag CMD	DATA[2]
0x9E	0x20	0x02	SN

Data[2] : Sector Number to be accessed.

For Type 3 Tag (Tag Type = 0x30)

- Polling, Tag CMD=0x00

Send Polling command to the T3T.

CMD	Tag Type	Tag CMD	Data[2].. Data[3]	Data[4]	Data[5]
0x9E	0x30	0x00	System Code	Time Slot	Request Flag

Data[2]...Data[3] : System Code (0xFFFF, 0x12FC)

Data[4] : Time Slot

Data[5] : System Code Request Flag, 0x00=no request, 0x01=request

- Check, Tag CMD=0x01

Read data from the specified Service Code and Block Lists.

CMD	Tag Type	Tag CMD	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]	Data[7]	...	Data[j]	Data[j+1]
0x9E	0x30	0x01	NS	NB	SC1L	SC1H	SC2L	SC2H	...	BL1H	BL1L

Data[2] : NS = Number of Services,

Data[3] : NB = the total Number of Blocks to be accessed

SCxL = Lower Byte of Service Code x,

SCxH = Higher Byte of Service Code x

Service Code consists of a Service Number(MSB 10bit of the 2bytes) and an Access Attribute(LSB 6bit of the 2bytes). Service Code should be sent in Little Endian format.

Access Attribute = 001001b (Read/Write)
= 001011b (Read Only)

BLxH : Higher Byte of Block x indicates Service Code List Order from 0x00 to 0x0F

The Service the block belongs to. The first BL1H should start from 0x00.

BLxL : Lower Byte of Block x indicates the Block Number of the Service Code

1 Block consists of 16 bytes of data.

To read data from Block 0, 1 of Service Code 0009h and Block 2,3 of Service Code 000Bh,

-> 9E 30 01 02 04 09 00 0B 00 00 00 00 01 01 02 01 03 should be sent to the reader

- Update, Tag CMD=0x02

Write data to the specified Service Code and Block Lists.

CMD	Tag Type	Tag CMD	Data[2]	Data[3]	Data[4]	Data[5]	Data[6]	Data[7]	...	Data[j]	Data[j+1]
0x9E	0x30	0x01	NS	NB	SC1L	SC1H	SC2L	SC2H	...	BL1H	BL1L

...	Data[k] ...
...	16 * NB

Data[2] : NS = Number of Services

Data[3] : NB = the total Number of Blocks to be accessed

SCxL = Lower Byte of Service Code x,

SCxH = Higher Byte of Service Code x

Service Code consists of a Service Number(MSB 10bit of the 2bytes) and an Access Attribute(LSB 6bit of the 2bytes). Service Code should be sent in Little Endian format.

Access Attribute = 001001b (Read/Write)
= 001011b (Read Only)

BLxH : Higher Byte of Block x indicates Service Code List Order from 0x00 to 0x0F

The Service the block belongs to. The first BL1H should start from 0x00.

BLxL : Lower Byte of Block x indicates the Block Number of the Service Code

To write data to Block 0 of Service Code 0009h and Block 2 of Service Code 000Bh,
-> 9E 30 02 02 02 09 00 0B 00 00 00 01 02 xx xx xx xx xx xx xx xx xx xx xx xx xx xx
yy yy yy yy yy yy yy yy yy yy yy yy yy yy yy yy should be sent to the reader

For Type 4 Tag (Tag Type = 0x40)

- Select File, Tag CMD=0x00

CMD	Tag Type	Tag CMD	DATA[2]
0x9E	0x40	0x00	SFlag

Data[2] : SFlag = 0x00 Select NDEF Application
= 0x01 Select CC File
= 0x02 Select NDEF File

- Read Binary, Tag CMD=0x01

CMD	Tag Type	Tag CMD	DATA[2]
0x9E	0x40	0x01	LEN

Data[2] : LEN = length of data to be read

- Update Binary, Tag CMD=0x02

CMD	Tag Type	Tag CMD	DATA[2]	Data[3] ... Data[n-1]
0x9E	0x40	0x01	LEN	DATA (LEN bytes)

Data[2] : LEN = length of data to be written

Data[3] ... Data[n-1] : Data to be written

- Response frame

For Type 1 Tag (Tag Type = 0x10)

- Read Identifier, Tag CMD = 0x00

Resp	Data[0]	Data[1]	DATA[2]	Data[3]	Data[4]	Data[5]
OK	HR0	HR1	UID0	UID1	UID2	UID3

-HRx : Header ROM Format

-UIDx : Unique Identifier

- Read All, Tag CMD = 0x01

Resp	Data[0]	Data[1]	DATA[2]	Data[3]	Data[4]	...	Data[128]	Data[129]
OK	HR0	HR1	UID0	UID1	UID2	...	xx	xx

-HRx : Header ROM Format

-UIDx : Unique Identifier

- Read a byte, Tag CMD = 0x02

Resp	Data[0]	Data[1]
OK	Byte Address	DATA

-Data[0] : Byte Address of the reading DATA

-Data[1] : DATA reading from a tag

- Write a byte, Tag CMD = 0x03 or 0x04

Resp	Data[0]	Data[1]
OK	Byte Address	DATA

-Data[0] : Byte Address of the written DATA

-Data[1] : DATA reading from a tag

For Type 2 Tag (Tag Type = 0x20),

- Read Block, Tag CMD=0x00

Resp	Data[0] ... Data[15]
OK	DATA(16bytes from the Block)

- Write Block, Tag CMD=0x01

Resp	Data[0]
OK	ACK(0x0A) or NACK

- Sector Select, Tag CMD=0x02

Resp	Data[0]
OK	ACK(0x0A) or NACK

For Type 3 Tag (Tag Type = 0x30)

- Polling, Tag CMD=0x00

Resp	Data[0]...Data[7]	Data[8]...Data[15]	Data[16]...Data[17]
OK	IDm	PMm	[System Code]

Optional Data : System Code when System Code is requested in the Polling Frame.

- Check, Tag CMD=0x01

Normal response from the Tag :

Resp	Data[0]	Data[1]~
OK	NB	16 x BN

Data[0] : NB=Number of Blocks, it indicates the total block number from reading.

Data[1]~.: Data from the Tag, Byte number of data after reading should be multiple of 16.

Error response from the Tag :

Resp	Data[0]	Data[1]
OK	Status1 Flag	Status2 Flag

Data[0] : Status1 Flag = other value except 0x00

Data[1] : Status2 Flag = other value except 0x00

- Update, Tag CMD=0x02

Normal response from the Tag :

Resp	Data[0]	Data[1]
OK	Status1 Flag	Status2 Flag

Data[0] : Status1 Flag = 0x00

Data[1] : Status2 Flag = 0x00

Error response from the Tag :

Resp	Data[0]	Data[1]
OK	Status1 Flag	Status2 Flag

Data[0] : Status1 Flag = other value except 0x00

Data[1] : Status2 Flag = other value except 0x00

- Return value

This function returns different values depending on whether it succeeds or fails.

Success : DE_OK.

Failure : An error code. For more information, refer to 10. Response code definition

14 Response code definitions

DE_OK	0
DE_NO_TAG_ERROR	2
DE_CRC_ERROR	3
DE_EMPTY	4
DE_AUTHENTICATION_ERROR	5
DE_NO_POWER	5
DE_PARITY_ERROR	6
DE_CODE_ERROR	7
DE_SERIAL_NUMBER_ERROR	8
DE_KEY_ERROR	9
DE_NOT_AUTHENTICATION_ERROR	10
DE_BIT_COUNT_ERROR	11
DE_BYTE_COUNT_ERROR	12
DE_TRANSFER_ERROR	14
DE_WRITE_ERROR	15
DE_INCREMENT_ERROR	16
DE_DECREMENT_ERROR	17
DE_READ_ERROR	18
DE_OVERFLOW_ERROR	19
DE_POLLING_ERROR	20
DE_FRAMING_ERROR	21
DE_ACCESS_ERROR	22
DE_UNKNOWN_COMMAND_ERROR	23
DE_ANTICOLLISION_ERROR	24
DE_INITIALIZATION_ERROR	25
DE_INTERFACE_ERROR	26
DE_ACCESS_TIMEOUT_ERROR	27
DE_NO_BITWISE_ANTICOLLISION_ERROR	28
DE_FILE_ERROR	29
DE_INVALID_BLOCK_ERROR	32
DE_ACK_COUNT_ERROR	33
DE_NACK_DESELECT_ERROR	34

DE_NACK_COUNT_ERROR	35
DE_SAME_FRAME_COUNT_ERROR	36
DE_RCV_BUFFER_TOO_SMALL_ERROR	49
DE_RCV_BUFFER_OVERFLOW_ERROR	50
DE_RF_ERROR	51
DE_PROTOCOL_ERROR	52
DE_USER_BUFFER_FULL_ERROR	53
DE_BUADRATE_NOT_SUPPORTED	54
DE_INVAILD_FORMAT_ERROR	55
DE_LRC_ERROR	56
DE_FRAMERR	57
DE_WRONG_PARAMETER_VALUE	60
DE_INVAILD_PARAMETER_ERROR	61
DE_UNSUPPORTED_PARAMETER	62
DE_UNSUPPORTED_COMMAND	63
DE_INTERFACE_NOT_ENABLED	64
DE_ACK_SUPPOSED	65
DE_NACK_RECEIVED	66
DE_BLOCKNR_NOT_EQUAL	67
DE_TARGET_SET_TOX	68
DE_TARGET_RESET_TOX	69
DE_TARGET_DESELECTED	70
DE_TARGET_RELEASED	71
DE_ID_ALREADY_IN_USE	72
DE_INSTANCE_ALREADY_IN_USE	73
DE_ID_NOT_IN_USE	74
DE_NO_ID_AVAILABLE	75
DE_OTHER_ERROR	76
DE_INVALID_READER_STATE	77
DE_MI_JOINER_TEMP_ERROR	78
DE_NOTYET_IMPLEMENTED	100
DE_FIFO_ERROR	109
DE_WRONG_SELECT_COUNT	114
DE_WRONG_VALUE	123

DE_VALERR	124
DE_RE_INIT	126
DE_NO_INIT	127
APP_INVALID_PORT	1000
APP_STX_ERROR	1001
APP_INVALID_LENGTH_ERROR	1002
APP_TIMEOUT_ERROR	1003
APP_CRC_ERROR	1004
APP_LRC_ERROR	1005
APP_RW_ERROR	1006
APP_ETX_ERROR	1007
APP_USB_WRITE_ERROR	1008
APP_USB_READ_ERROR	1009
APP_INVALID_SENDDATA_LEN	1010
APP_INVALID_SENDBUF_SIZE	1011
APP_TOO_SMALL_RECVBUF	1012
APP_SENDBUF_OVERFLOW	1013

15 Time out value

TOUT Value in hexadecimal	Time Value	FWI
0x02	1.208 msec	2
0x03	2.416 msec	3
0x05	4.832 msec	4
0x0A	9.664 msec	5
0x14	19.32 msec	6
0x27	38.66 msec	7
0x4E	77.3 msec	8
0x9B	154.6 msec	9
0xB0 ~	309.3 msec	10
0xC0 ~	618.6 msec	11
0xD0 ~	1.2371 sec	12
0xE0 ~	2.4742 sec	13
0xF0 ~	4.9485 sec	14
0xFF	9.897 sec	15